Flapping Butterfly Clip

Build and Concepts Guide (ESP32-C3 + 0.42” OLED)

Electronics-AndresForero

October 14, 2026

Contents

1 What you are building **1**

2 Safety first **2**

3 Parts **2**

4 Concepts **2**
4.1 How a hobby servo is controlled . . . 2
4.2 Easing . . . 2
4.3 Non-blocking code . . . 2
4.4 Debouncing . . . 3
4.5 Voltage divider and the ADC . . . 3
4.6 Why a boost converter (and when to skip it) . . . 3
4.7 Servo supply rule . . . 3
4.8 Low-battery latch . . . 3

5 Wiring **3**
5.1 Phase 1 . . . 3
5.2 Phase 2 . . . 3

6 Firmware walkthrough **4**

7 Build, flash, test **4**

8 Troubleshooting **5**

1 What you are building

A hair clip with butterfly wings that flap when you touch it. The same firmware runs in two phases:

Phase 1, bench
: USB powered, push-button, any 5 V micro servo. Goal: make the code and mechanism work.

Phase 2, wearable
: Single-cell LiPo, 2 g servo, TTP223 touch module, battery monitor. Goal: small, light, safe.

The ESP32-C3 board has a built-in 72×40 pixel OLED that shows the state, battery voltage and servo angle.

Credit. Inspired by the *Flapping Butterfly Clip* by **@resetwithregine** on TikTok and the *Flapping Butterfly Clip v2 Upgrades* build guide on Notion, who also published the v1 gear mechanism. The firmware, CAD and this document are an independent rewrite; no STL files, code or images from the original are reused. Please support the original creator, and credit them and this repository if you build on this work.

2 Safety first

- Use a LiPo with a **protection PCB**. Never use a bare cell.
- Add a **hard power switch**. Stopping the wings does *not* disconnect the battery.
- Charge **off the clip**, at or below the cell's rated current, on a non-flammable surface, never unattended.
- Insulate every joint with heat-shrink. Keep electronics off the metal clip and your skin.
- Enclose the gears. Keep hair and fingers out of the mechanism; disconnect the battery before changing wings.
- Check the boost converter output with a multimeter *before* connecting the board.

3 Parts

The authoritative list is `hardware/BOM.md`. In short:

Part	Phase 1	Phase 2
Board	ESP32-C3 + 0.42" OLED	same
Servo	SG92R / MG90S (5 V)	2 g, 3.7–4.2 V
Power	USB	1S LiPo 150–300 mAh (+ MT3608)
Trigger	push-button	TTP223 touch module
Extras	470 µF cap	switch, divider 2×100 k, cap, 28–30 AWG wire

4 Concepts

4.1 How a hobby servo is controlled

A servo expects a pulse every 20 ms (50 Hz). The *width* of that pulse sets the angle: about 0.5 ms is 0° and about 2.4 ms is 180° (values in `config.h`). The ESP32Servo library generates this with the chip's hardware PWM (LEDC), so it costs no CPU time.

4.2 Easing

Jumping straight to a new angle is jerky. We move along

$$e(t) = \tfrac{1}{2} - \tfrac{1}{2}\cos(\pi t), \qquad t \in [0, 1],$$

which accelerates then decelerates. The angle is $\theta = \theta_0 + (\theta_1 - \theta_0)\, e(t)$.

4.3 Non-blocking code

`delay()` freezes the program. Instead, every loop iteration reads `millis()`, computes where the wing *should* be at that instant, and moves on. That is why touch, battery and display keep working during a flap.

4.4 Debouncing

Mechanical contacts chatter for a few milliseconds. A new level is accepted only after it stays unchanged for `DEBOUNCE_MS`, and one event is produced per press.

4.5 Voltage divider and the ADC

A LiPo reaches 4.2 V but the ADC tolerates about 2.5 V. Two equal resistors give $V_{pin} = V_{bat}\, R_2/(R_1 + R_2) = V_{bat}/2$. The firmware multiplies the reading by 2. Calibrate with a multimeter using `VBAT_CAL`.

4.6 Why a boost converter (and when to skip it)

The board's own regulator needs about 3.5 V or more to hold 3.3 V. As the cell sags, the boost converter keeps the board stable down to about 3.0 V and lets you use more of the capacity, at the price of one extra part, about 2 g, and roughly 15 % conversion loss. With a 3.7–4.2 V servo you may skip it, connect the cell straight to `V5`, and raise `VBAT_CUTOFF_MV` to 3500.

4.7 Servo supply rule

A 3.7–4.2 V servo is powered from the *battery*; a 5 V servo from the *boosted* rail. Connecting the wrong one either under-drives the servo or damages it. Grounds are always common.

4.8 Low-battery latch

Under load the cell voltage dips. The firmware low-pass filters the reading, requires several consecutive low samples, and then *latches* the fault until reboot, so the wings do not restart the moment the voltage recovers. The cell's own protection PCB cuts at about 2.5 V, far too low to be relied upon.

5 Wiring

Check the pins first: flash `hwcheck` and confirm the OLED answers at `0x3C` on `GPIO5`/`GPIO6`.

Signal	Pin	Note
Servo signal	`GPIO3`	
Trigger	`GPIO4`	button to GND (P1) / TTP223 OUT (P2)
Battery sense	`GPIO0`	ADC1, Phase 2
OLED SDA/SCL	`GPIO5`/`GPIO6`	on-board, do not use
Avoid	`GPIO2`, `GPIO8`, `GPIO9`	boot strapping pins

5.1 Phase 1

Servo red to `V5`, brown to `GD`, signal to `GPIO3`; button between `GPIO4` and `GD`; 470 µF across `V5`/`GD`.

5.2 Phase 2

1. Battery + → slide switch → node `VBAT`. Battery − is common GND.
2. `VBAT` → MT3608 `IN+`. Set the output to 5.0 V with a multimeter, then `OUT+` → board `V5`.
3. `VBAT` → servo red (3.7–4.2 V servo). Servo brown to GND. Signal to `GPIO3`.
4. `VBAT` → 100 k → `GPIO0` → 100 k → GND.
5. TTP223: VCC to `V3`, GND to GND, OUT to `GPIO4`.

6. 100–470 µF across the MT3608 output.

6 Firmware walkthrough

File	Role
`include/config.h`	every pin and tunable, build-variant switches
`include/WingServo.h`	eased, non-blocking flapping; detaches when idle
`include/TouchButton.h`	debounced press detection for both input types
`include/Battery.h`	divider reading, filter, percent, latched low flag
`include/StatusDisplay.h`	four-line OLED status
`src/main.cpp`	state machine
`tools/hwcheck/main.cpp`	I2C scan and ADC check

State machine. `IDLE` $\xrightarrow{\text{touch}}$ `FLAPPING` $\xrightarrow{\text{touch}}$ `IDLE`; any state $\xrightarrow{\text{low battery}}$ `LOW_BATT` (latched, Phase 2).

The core of the flap (from `WingServo.h`):

```
float t = float(now - t0_) / float(FLAP_MS);    // 0..1 progress of this stroke
float e = 0.5f - 0.5f * cosf(PI * t);           // sine ease in-out
angle_  = from_ + (to_ - from_) * e;            // interpolate between the ends
servo_.write(int(angle_ + 0.5f));
```

7 Build, flash, test

```
pio run -e hwcheck -t upload && pio device monitor   # confirm pins
pio run -e bench    -t upload                        # Phase 1
pio run -e wearable -t upload                        # Phase 2
```

1. **hwcheck:** an I2C device at `0x3C` on `GPIO5`/`GPIO6`. If not, fix `config.h` before continuing.
2. **bench, no servo:** OLED shows IDLE; each press toggles FLAPPING and the angle number moves.
3. **bench, with servo:** wings flap smoothly; if gears bind, lower `ANGLE_CLOSED_DEG`.
4. **wearable, on a bench supply or the 2000 mAh cell:** compare the displayed voltage with a multimeter, then set `VBAT_CAL`.
5. **Cutoff:** run from a supply you can lower slowly; confirm LOW BATT appears near the cutoff and wings stop.
6. **Runtime:** measure the real current. The estimate (1–1.5 h from 250 mAh) is unverified.

8 Troubleshooting

Symptom	Likely cause
Board resets when the servo moves	weak supply; add/enlarge the capacitor, shorten wires
OLED blank	wrong pins; run `hwcheck`
Servo buzzes at the end stops	travel too large; reduce `ANGLE_CLOSED_DEG`
Battery reads wrong	divider values or `VBAT_CAL`; check with a multimeter
Touch toggles by itself	TTP223 floating or sharing a long wire; keep it short and grounded